# 2.4. Методы

**К концу занятия ты сможешь:**

- написать свой метод и вызывать его сколько угодно раз;
- передавать в метод данные через параметры;
- получать из метода ответ через return;
- объяснить, где «живёт» переменная, и делить работу между методами.

# Разминка: темы 2.1–2.3

**Вопрос 1**

Что выведет программа?

Program.cs

```
int[] marks = { 5, 4, 3 };
Console.WriteLine(marks[1]);
Console.WriteLine(marks.Length);
```

**Вопрос 2**

Что выведет программа и почему?

Program.cs

```
string word = "привет";
word.ToUpper();
Console.WriteLine(word);
```

**Вопрос 3**

Как получить последний элемент массива marks, не зная заранее, сколько в нём элементов?

# Задача: полоска здоровья в игре

В игре полоска здоровья рисуется в трёх местах: в начале боя, после удара и после зелья.

Начало боя → цикл рисует полоску

Враг ударил → тот же цикл, копия 2

Герой выпил зелье → тот же цикл, копия 3

Дизайнер просит рисовать полоску знаком **=** вместо **#**. Сколько мест придётся править? А если их будет двадцать?

**Как сделать, чтобы правка была одна?**

Вывод в консоли

```
Начало боя
Здоровье: [##########] 100
Враг ударил
Здоровье: [######] 60
Герой выпил зелье
Здоровье: [#########] 90
```

# План занятия

## Урок 1

- Разминка и задача про полоску здоровья
- Блок 1. Свой метод: void, вызов, параметры
- Вопросы по блоку 1
- Практика «Отряд героев»

## Урок 2

- Блок 2. Результат метода: return, где живёт переменная, один метод — одна задача
- Вопросы и практика «Калькулятор скидки»
- Мини-проект «Тренажёр счёта»
- Итоги

# Вспомним: цикл for (1.5) и Length (2.1, 2.3)

Program.cs

```
string stars = "";
for (int i = 0; i < 4; i++)
{
    stars += "*";
}
Console.WriteLine(stars + " " + stars.Length);
```

Вывод в консоли

```
**** 4
```

- for повторяет тело цикла 4 раза: i = 0, 1, 2, 3.
- stars += "*" дописывает звёздочку в конец строки.
- Length — длина строки, у массива — число элементов.
- Сегодня этот цикл будет рисовать полоску здоровья.

Блок 1 из 2 • Урок 1

# Свой метод

**В этом блоке ты научишься:**

- объявлять метод void и вызывать его по имени;
- понимать, в каком порядке выполняются строки;
- передавать в метод данные через параметры.

# Что такое метод и зачем он нужен

**Метод** — кусок кода с именем. Его записывают один раз, а запускают по имени сколько угодно раз.

Как рецепт в тетради: он записан один раз. Чтобы испечь блины снова, рецепт не переписывают — его открывают.

**Без методов** одинаковые строки копируются. Правку или ошибку приходится искать в каждой копии.

Program.cs — без метода

```
Console.WriteLine("==============");
Console.WriteLine("Новый раунд");
Console.WriteLine("Игрок бросает кубик...");
Console.WriteLine("==============");
Console.WriteLine("Новый раунд");
Console.WriteLine("Игрок бросает кубик...");
```

Вывод в консоли

```
==============
Новый раунд
Игрок бросает кубик...
==============
Новый раунд
Игрок бросает кубик...
```

# Как записывается метод

Шаблон

```
static void PrintBanner()
{
    // команды метода
}
```

**Где писать в Program.cs:**

Сверху — основной код. Выполняется строка за строкой.

Ниже — методы. Работают, только когда их вызовут.

| Часть | Что значит |
|---|---|
| static | Пока просто запомни: пишем перед каждым методом. Подробно — в теме 4.3 |
| void | Метод ничего не возвращает, просто выполняет команды |
| PrintBanner | Имя метода: с большой буквы, говорит, что метод делает |
| ( ) | Скобки для параметров. Пока пустые |
| { } | Тело метода — команды, которые выполнятся при вызове |

# Первый метод: объявить и вызвать

Program.cs

```
PrintBanner();
Console.WriteLine("Игрок бросает кубик...");
PrintBanner();
Console.WriteLine("Игрок бросает кубик...");

static void PrintBanner()
{
    Console.WriteLine("==============");
    Console.WriteLine("Новый раунд");
}
```

Вывод в консоли

```
==============
Новый раунд
Игрок бросает кубик...
==============
Новый раунд
Игрок бросает кубик...
```

**Объявить** — написать метод ниже основного кода.

**Вызвать** — написать имя, скобки и точку с запятой: PrintBanner();

# Вызов: куда «прыгает» программа

Program.cs

```
Console.WriteLine("1. Начало программы");
SayHello();
Console.WriteLine("3. Конец программы");

static void SayHello()
{
    Console.WriteLine("2. Внутри метода");
}
```

Вывод в консоли

```
1. Начало программы
2. Внутри метода
3. Конец программы
```

| Шаг | Строка | Что происходит |
|---|---|---|
| 1 | 1 | Выводится «1. Начало программы» |
| 2 | 2 | Вызов SayHello(): переход в метод |
| 3 | 7 | Выводится «2. Внутри метода» |
| 4 | 8 | Метод закончился: возврат к строке после вызова |
| 5 | 3 | Выводится «3. Конец программы» |

**После метода программа возвращается туда, откуда его вызвали.**

# Параметры: метод получает данные

**Параметр** — переменная метода. В неё при вызове кладётся значение.

**Аргумент** — само значение, которое передаём при вызове.

Как кофемашина: кнопка одна, а какой напиток сделать, говоришь ты.

`PrintBanner("Раунд 1")` → `title = "Раунд 1"`

Program.cs

```
PrintBanner("Раунд 1");
PrintBanner("Раунд 2");

static void PrintBanner(string title)
{
    Console.WriteLine("==============");
    Console.WriteLine(title);
}
```

Вывод в консоли

```
==============
Раунд 1
==============
Раунд 2
```

# Несколько параметров

Program.cs

```
PrintScore("Алиса", 120);
PrintScore("Борис", 95);

static void PrintScore(string playerName, int score)
{
    Console.WriteLine(playerName + ": " + score + " очков");
}
```

Вывод в консоли

```
Алиса: 120 очков
Борис: 95 очков
```

- Параметры пишем через запятую, у каждого свой тип.
- Аргументы попадают в параметры по порядку: первый в первый, второй во второй.

# Ловушка: аргументы не в том порядке

**Ошибка**

Program.cs (метод PrintScore — с прошлого слайда)

```
PrintScore(120, "Алиса");
```

Сообщение компилятора

```
error CS1503: Argument 1: cannot convert from 'int' to 'string'
Аргумент 1: нельзя преобразовать int в string
error CS1503: Argument 2: cannot convert from 'string' to 'int'
Аргумент 2: нельзя преобразовать string в int
```

Первый параметр — string, а пришло число. Программа не запустится, пока не исправишь.

**Правильно**

Program.cs

```
PrintScore("Алиса", 120);
```

Вывод в консоли

```
Алиса: 120 очков
```

Место для скриншота: красное подчёркивание и окно «Список ошибок»

# Живой кодинг: полоска здоровья в методе

1. Основной код: health = 100, вывод «Начало боя».
2. Ниже — метод static void PrintHealthBar(int health): цикл for собирает полоску, один # на 10 очков.
3. Вызываем PrintHealthBar(health) после «Начало боя».
4. Ошибка нарочно: вызываем PrintHealthBar() без аргумента. Читаем сообщение и исправляем.
5. Добавляем удар (−40) и зелье (+30), вызываем метод ещё два раза.
6. Запускаем и сверяем вывод.

Вывод в консоли

```
Начало боя
Здоровье: [##########] 100
Враг ударил
Здоровье: [######] 60
Герой выпил зелье
Здоровье: [#########] 90
```

# Правка в одном месте

Дизайнер просит знак = вместо #. Меняем одну строку в методе PrintHealthBar — меняются все три полоски.

**Было**

В методе PrintHealthBar

```
bar += "#";
```

Вывод в консоли

```
Здоровье: [##########] 100
Здоровье: [######] 60
Здоровье: [#########] 90
```

**Стало**

В методе PrintHealthBar

```
bar += "=";
```

Вывод в консоли

```
Здоровье: [==========] 100
Здоровье: [======] 60
Здоровье: [=========] 90
```

**Правило: повторяющийся код выносим в метод — пишем один раз, вызываем сколько нужно, правим в одном месте.**

# Вопросы по блоку 1

Program.cs (ниже основного кода)

```
static void Greet(string name)
{
    Console.WriteLine("Привет, " + name);
}
```

1. Метод Greet написан, но в основном коде его ни разу не вызвали. Что он выведет?
2. Что выведут вызовы Greet("Мира"); и Greet("Тимур");? Где здесь параметр, а где аргументы?
3. Найди ошибку в каждом вызове: Greet(); и PrintScore(95, "Борис");

# Задание: «Отряд героев»

- Напиши метод PrintHero(string name, int health). Он выводит две строки: «Герой: имя» и «Здоровье: N из 100».
- Если здоровье меньше 40, метод выводит третью строку: «Нужен отдых».
- Основной код выводит «Отряд готов к походу» и вызывает метод для трёх героев: Рыцарь 80, Лучница 55, Маг 30.

**Подсказка:** метод пиши ниже основного кода. if внутри метода работает так же, как в основном коде (1.4).

★ **Успел раньше?** Добавь в метод полоску из # — один знак на каждые 10 очков, как в живом кодинге.

Застрял — 15 минут ищешь сам, потом спрашиваешь: что делал, что ожидал, что получил.

# Что должно получиться

Вывод в консоли

```
Отряд готов к походу
Герой: Рыцарь
Здоровье: 80 из 100
Герой: Лучница
Здоровье: 55 из 100
Герой: Маг
Здоровье: 30 из 100
Нужен отдых
```

## Проверь себя

- В основном коде три вызова метода, а не три копии строк.
- «Нужен отдых» выводится только у Мага.
- Ты можешь объяснить каждую строку своего метода.

# ★★ Ромб любого размера

Напиши метод PrintDiamond(int size): он рисует ромб из звёздочек, у которого до середины size строк. Вызови его для 2, 3 и 4.

**Подсказка:** каждая строка ромба — пробелы, потом звёздочки. Удобно сделать второй метод PrintRow(int spaces, int stars).

PrintDiamond(2)

```
 *
***
 *
```

PrintDiamond(3)

```
  *
 ***
*****
 ***
  *
```

PrintDiamond(4)

```
   *
  ***
 *****
*******
 *****
  ***
   *
```

Блок 2 из 2 • Урок 2

# Результат метода

**В этом блоке ты научишься:**

- возвращать из метода ответ через return и пользоваться им;
- понимать, где «живёт» переменная;
- делить работу между методами: один метод — одна задача.

# return: метод отдаёт ответ

Метод void только выполняет команды. Метод с результатом ещё и **отдаёт ответ** туда, где его вызвали.

Как калькулятор: набрал пример — получил число, а что с ним делать, решаешь сам.

| Запись | Что значит |
| --- | --- |
| int вместо void | Тип ответа: метод вернёт целое число |
| return значение; | Отдать ответ и сразу выйти из метода |

Program.cs

```
int result = Square(7);
Console.WriteLine(result);

static int Square(int number)
{
    return number * number;
}
```

Вывод в консоли

```
49
```

# Что делать с ответом метода

Program.cs

```
int a = Square(3);
int b = Square(4) + Square(5);
Console.WriteLine(Square(6));
Console.WriteLine(a);
Console.WriteLine(b);

static int Square(int number)
{
    return number * number;
}
```

Вывод в консоли

```
36
9
41
```

- Сохранить в переменную: a = Square(3)
- Использовать в выражении: Square(4) + Square(5)
- Сразу передать в другой метод: Console.WriteLine(Square(6))

**Вызов метода с ответом int можно писать везде, где можно написать число int.**

# Ловушка: ответ метода потерялся

Метод Square — со слайда «return: метод отдаёт ответ».

**Ошибка**

Program.cs

```
Console.WriteLine("Квадрат числа 7:");
Square(7);
```

Вывод в консоли

```
Квадрат числа 7:
```

**Правильно**

Program.cs

```
Console.WriteLine("Квадрат числа 7:");
Console.WriteLine(Square(7));
```

Вывод в консоли

```
Квадрат числа 7:
49
```

Square(7) посчитал 49 и вернул, но ответ никто не взял. Visual Studio не подчеркнёт ошибку — программа просто промолчит. Так же было с word.ToUpper() в разминке.

# Где «живёт» переменная

Program.cs

```
CountDown();
Console.WriteLine("Осталось: " + secondsLeft);

static void CountDown()
{
    int secondsLeft = 3;
    Console.WriteLine("Старт через " + secondsLeft);
}
```

**Основной код**

secondsLeft — не видно

**Метод CountDown { стены }**

```
int secondsLeft = 3;
```

Переменная, объявленная в методе, живёт только внутри его { }. Метод закончился — переменной нет.

Сообщение компилятора

```
error CS0103: The name 'secondsLeft' does not exist in the current context
Имени secondsLeft нет в текущем месте программы
```

**Нужно значение снаружи — верни его через return. Нужно внутри — передай параметром.**

# Один метод — одна задача

Program.cs — метод и считает, и печатает

```
PrintTotal(12, 30, 25);

static void PrintTotal(int roundOne, int roundTwo, int roundThree)
{
    int total = roundOne + roundTwo + roundThree;
    Console.WriteLine("Итого: " + total + " очков");
}
```

Вывод в консоли

```
Итого: 67 очков
```

**В чём проблема:** сумма заперта внутри метода. Её нельзя сравнить с рекордом или вывести по-другому.

**Правило: один метод считает и возвращает, другой выводит. Имя говорит, что метод делает.**

# Живой кодинг: делим метод на два

1. Пишем метод static int CalculateTotal(int roundOne, int roundTwo, int roundThree) — он только считает.
2. Ошибка нарочно: забываем return. Читаем сообщение и дописываем return.
3. Меняем PrintTotal: теперь он получает готовую сумму, PrintTotal(int total), и только печатает.
4. В основном коде: int total = CalculateTotal(12, 30, 25);
   и PrintTotal(total);
5. Добавляем то, что раньше было невозможно: если total больше 60 — «Новый рекорд».

Вывод в консоли

```
Итого: 67 очков
Новый рекорд
```

# Вопросы по блоку 2

Program.cs (ниже основного кода)

```
static int Square(int number)
{
    return number * number;
}

static void PrintSquare(int number)
{
    Console.WriteLine(number * number);
}
```

1. Что выведет каждая строка: Square(5); и Console.WriteLine(Square(Square(2)));?
2. Почему не компилируется строка int result = PrintSquare(7);?
3. В методе CalculateTotal объявлена переменная sum. Можно ли вывести sum в основном коде? Почему?

# Задание: «Калькулятор скидки»

- Программа спрашивает цену товара и скидку в процентах.
- Метод GetFinalPrice(decimal price, int discount) считает и возвращает цену со скидкой. Сам он ничего не выводит.
- Основной код выводит «Цена со скидкой: …».

**Подсказка:** цена со скидкой = цена − цена × скидка / 100. Округли до копеек: Math.Round(result, 2) (1.3). Дробную цену вводи через запятую: 1999,90.

★ **Успел раньше?** Метод bool IsDiscountValid(int discount) проверяет, что скидка от 0 до 100. Если нет — программа пишет «Скидка должна быть от 0 до 100» и цену не считает.

# Что должно получиться

Вывод в консоли — пример 1

```
Цена товара: 1999,90 <- ввёл пользователь
Скидка в процентах: 20 <- ввёл пользователь
Цена со скидкой: 1599,92
```

Вывод в консоли — пример 2

```
Цена товара: 450 <- ввёл пользователь
Скидка в процентах: 15 <- ввёл пользователь
Цена со скидкой: 382,5
```

Стрелка — пометка для тебя, в программе её нет.

## Проверь себя

- В методе GetFinalPrice нет Console.WriteLine — он только возвращает ответ.
- При скидке 0 цена не меняется.
- Цену вводишь через запятую: 1999,90, а не 1999.90.

# ★★ Сколько простых чисел?

Простое число делится только на 1 и на само себя: 2, 3, 5, 7, 11…

Напиши метод CountPrimes(int limit), который возвращает, сколько простых чисел от 2 до limit. Проверку одного числа вынеси в отдельный метод bool IsPrime(int number). Вызови CountPrimes для 10, 100 и 1000.

Вывод в консоли

```
Простых чисел до 10: 4
Простых чисел до 100: 25
Простых чисел до 1000: 168
```

# Вопросы по занятию

1. Вспомни 2.1: как будет выглядеть заголовок метода, который получает массив оценок и возвращает их среднее? Какой тип у параметра и у ответа?
2. Что изменится в выводе живого кодинга, если в PrintHealthBar заменить health / 10 на health / 20?
3. Сколько раз выполнится цикл for (1.5) при вызове PrintHealthBar(60)? А если вызвать метод три раза подряд с тем же числом?

# Мини-проект: «Тренажёр счёта»

Программа задаёт 10 примеров на сложение двузначных чисел, проверяет ответы и в конце выдаёт звание.

Вывод в консоли — начало игры

```
Пример 1: 44 + 88 = 132 <- ввёл пользователь
Верно
Пример 2: 69 + 14 = 83 <- ввёл пользователь
Верно
Пример 3: 42 + 70 = 111 <- ввёл пользователь
Неверно, правильно: 112
```

Вывод в консоли — конец игры

```
Пример 8: 89 + 46 = абв <- ввёл пользователь
Неверно, правильно: 135
Пример 9: 82 + 40 = 122 <- ввёл пользователь
Верно
Пример 10: 88 + 12 = 100 <- ввёл пользователь
Верно
Правильных ответов: 7 из 10
Звание: Знаток
```

Стрелка — пометка для тебя, в программе её нет. Примеры 4–7 пропущены. Числа каждый раз новые — их выбирает Random.

# Что должна уметь программа

- Задаёт ровно 10 примеров, числа каждый раз новые — от 10 до 99.
- После каждого ответа пишет «Верно» или «Неверно, правильно: …».
- Если ввести буквы вместо числа, не падает, а засчитывает ответ как неверный.
- В конце пишет число правильных ответов и звание: 9–10 — «Математик», 6–8 — «Знаток», 0–5 — «Новичок».

Это пригодится в Проекте 2 «Игра на поле»: там код тоже делится на методы — отрисовка, ход игрока, проверка победы.

# План по шагам

| Шаг | Что сделать | Темы | Как проверить |
|---|---|---|---|
| 1 | Допиши метод AskQuestion: прочитай ответ через int.TryParse и верни, равен ли он a + b | 1.2, 2.4 | Временно напиши Console.WriteLine(AskQuestion(1, 2, 3)); На ответ 5 — True, на 4 — False |
| 2 | Цикл на 10 примеров: a и b через random.Next(10, 100), вызов AskQuestion, «Верно» или «Неверно, правильно: …», счёт в score | 1.5, 1.4 | Запусти: ровно 10 примеров, числа от 10 до 99 |
| 3 | Метод PrintResult(int score): число правильных ответов и звание | 1.4, 2.4 | Ответь на всё верно — «Правильных ответов: 10 из 10» и «Звание: Математик» |

# Заготовка

Program.cs

```
Random random = new Random();
int score = 0;
for (int i = 1; i <= 10; i++)
{
    // Шаг 2: a и b - случайные от 10 до 99, вызов AskQuestion,
    // при верном ответе score++ и "Верно", иначе "Неверно, правильно: ..."
}
// Шаг 3: вызови PrintResult(score) и напиши этот метод ниже

static bool AskQuestion(int number, int a, int b)
{
    Console.Write("Пример " + number + ": " + a + " + " + b + " = ");
    return false; // Шаг 1: прочитай ответ через int.TryParse и верни answer == a + b
}
```

Заготовка запускается сразу. Будь готов объяснить каждую строку своего кода. Застрял — 15 минут ищешь сам, потом спрашиваешь.

# Готово, если…

- Блок 1: вопрос задаёт метод AskQuestion, итог печатает метод PrintResult — в основном коде нет повторов.
- Блок 2: AskQuestion возвращает true или false и сам не пишет «Верно».
- Если ввести «абв», программа пишет «Неверно, правильно: …» и задаёт следующий пример.
- Примеров ровно 10; на 10 верных ответов — «Звание: Математик».

### ★ Уровни сложности

В начале программа спрашивает «Сложность (1 - лёгкая, 2 - трудная):». На лёгкой числа от 1 до 9: «Пример 1: 4 + 7 = ».

### ★★ Случайное действие

Пример может быть на +, – или *. Метод GetCorrectAnswer(int a, int b, int operation) возвращает правильный ответ: «Пример 3: 14 * 6 = ».

# Турнир из трёх попыток

- Вынеси всю игру в метод int PlayRound(): он задаёт 10 примеров и возвращает счёт.
- Сыграй 3 попытки подряд, очки каждой сохрани в массив int[] (2.1).
- В конце выведи очки всех попыток, лучшую попытку и звание за неё.
- Звание выдаёт метод string GetRank(int score): он возвращает слово, а не печатает его.

Основной код должен получиться короче 15 строк. Остальное — в методах.

# Итоги

**Сегодня я научился:**

- выносить повторяющийся код в метод и вызывать его по имени;
- передавать в метод данные через параметры в правильном порядке;
- получать из метода ответ через return и не терять его;
- понимать, где видна переменная, и делить работу: один метод — одна задача.

**В следующий раз:**

перегрузка методов, необязательные и именованные параметры, params.